AI-BASED HEART DISEASE PREDICTION SYSTEM

End-to-End Testing, Methodology, Model Performance and User Guide

Project	AI-Based Heart Disease Prediction System for Clinical Decision Support
Application name	CardioAI
ML algorithm	Random Forest Classifier
Application stack	Python, Flask, scikit-learn, joblib, pandas, HTML and CSS
Document date	26 October 2026

Document status: methodology and model metrics are based on the project results shared during development. The application input page was observed locally, but a later browser screenshot showed a connection-refused error. End-to-end success is therefore documented as pending verification rather than claimed as complete.

Project documentation • Prepared for final deliverable

1. Executive Summary

This project explores a machine-learning approach to estimating the probability of heart disease from structured clinical features. A Random Forest classifier was tuned using GridSearchCV with five-fold stratified cross-validation. A Flask application named CardioAI is intended to collect clinical inputs and display probabilities for both No Heart Disease and Heart Disease classes.

The model evaluation results reported during development were 88.89% accuracy, 85.19% precision, 92.00% recall, 88.46% F1-score, and 0.9566 ROC-AUC on a 54-record held-out test set. These are experimental results on a small dataset and are not evidence of clinical readiness.

The UI was observed in a browser at the local Flask address, but the later screenshot showed **ERR_CONNECTION_REFUSED**. That error typically means the local server was not accepting connections. Because a successful submission and results page were not confirmed in the supplied evidence, the end-to-end test status is marked **pending** until retested.

2. Project Objectives

• Prepare clinical data for supervised learning.

• Train and tune a Random Forest classifier.

• Evaluate the final model using classification and ranking metrics.

• Integrate the saved model into a Flask-based web interface.

• Display class probabilities for both target classes.

• Document setup, operation, testing, limitations, and responsible use.

3. Dataset and Methodology

The dataset used for model development contained 297 records and 14 columns, including 13 input features and one target column. The reported class counts were 160 records with target 0 (No Disease) and 137 with target 1 (Heart Disease). The input features were age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, and thal.

The workflow was: data inspection and cleaning; feature preparation; stratified train/test split; Random Forest training; hyperparameter tuning with GridSearchCV; held-out test evaluation; model serialization; and intended Flask integration. Exact preprocessing must remain consistent between training and web inference.

Setting	Reported configuration
Dataset size	297 records; 13 features plus target
Test split	test_size=0.18, random_state=11, stratify=y
Tuning	GridSearchCV; 144 parameter combinations; 5-fold StratifiedKFold; 720 fits
Scoring metric	Recall
Best CV recall	0.7688 (76.88%)

4. Final Random Forest Configuration

The reported best hyperparameters were: criterion=gini, max_depth=None, max_features=sqrt, min_samples_leaf=1, min_samples_split=2, and n_estimators=100. The model artifact was previously reported at models/rf_heart_model.pkl. Confirm that this file exists in the final repository and that the application loads the same artifact used for evaluation.

5. Model Performance

Metric	Reported result
Accuracy	88.89%
Precision	85.19%

Metric	Reported result
Recall	92.00%
F1-score	88.46%
ROC-AUC	0.9566
Best cross-validation recall	76.88%

Confusion matrix (rows: actual class 0/1; columns: predicted class 0/1):

```
[[25, 4],
[ 2, 23]]
```

This corresponds to 25 true negatives, 4 false positives, 2 false negatives, and 23 true positives. The test set contained 54 records. The gap between cross-validation recall (76.88%) and test recall (92.00%) does not by itself prove overfitting; train-versus-validation comparison and leakage checks are needed.

6. Probability-Based Prediction

The web app is intended to use the trained classifier's predict_proba() method. The returned values must be mapped using the classifier's classes_ attribute so class 0 is displayed as No Heart Disease and class 1 as Heart Disease. For a binary classifier, the two values should sum to approximately 100% before rounding.

These are model-estimated class probabilities. Unless probability calibration and external validation are performed, they must not be described as a patient's validated clinical risk.

7. Web Application Architecture

The intended architecture consists of an HTML form, Flask request handler, saved Random Forest artifact, and results template. The form collects 13 features; the backend validates values and preserves the exact feature order used during training; the model returns class probabilities; and the results page displays both percentages with a gauge or progress bars.

Browser form	Flask backend	Saved model	Results page
Clinical inputs	Validate and order features	predict_proba()	Two percentages + gauge

8. End-to-End Testing Plan and Status

End-to-end testing should cover the full path from browser input to model output. Based on the evidence provided, the input page was visible, but a later request to localhost was refused. The following tests are required before the application can be marked as successfully integrated.

Test	Expected result	Status
Start Flask server	Server reports local URL and stays running	Needs retest
Open input page	Form loads without server error	Previously observed
Load CSS	Stylesheet URL returns CSS; page is styled	Needs retest
Load model	Model loads without exception	Needs verification
Submit valid values	Results page displays both probabilities	Pending
Check probability sum	Two probabilities total ~100%	Pending
Invalid/missing inputs	Clear validation feedback, no crash	Pending
Feature encoding	Input values match training encodings/order	Must verify

9. User Guidelines

1. Start the application from the project root using `python app.py` and keep the terminal open.

2. Open `http://127.0.0.1:5000/` after the terminal confirms the server is running.

3. Enter values taken from a reliable clinical record. Check units and categorical options carefully.

4. Submit the form and review both class probabilities. Verify that the result page loads and the values sum to approximately 100%.

5. If the browser reports ERR_CONNECTION_REFUSED, restart/check the Flask process and read the terminal traceback. If the CSS is missing, verify the default folder path `static/style.css`.

6. Use the result only as an educational/model demonstration, not as a diagnosis or treatment recommendation.

10. Limitations and Responsible Use

• The dataset contains 297 records; results may not generalize to other populations.

• The held-out test set is small (54 records), so metrics may vary across splits.

• No claim of clinical validation, external validation, or probability calibration is made.

• Incorrect feature order or category encodings can invalidate predictions.

• A web prediction must never delay urgent care or replace qualified medical assessment.

11. Conclusion and Final Deliverables

The model-development results are documented and the Flask UI has been observed locally, but successful end-to-end integration remains pending until the server is restarted and a valid form submission displays both class probabilities. The final deliverable should include the trained model artifact, Flask source code, HTML templates, CSS, dependency list, this report, and evidence of passing tests.

Recommended completion sequence: correct the static directory/file name; start Flask; verify model loading; run a valid prediction test; confirm probability mapping and sum; test invalid inputs; record test outcomes; then commit the verified application and report to the designated Git branch.

Medical disclaimer: CardioAI is a research/educational prototype. It is not a medical device and must not be used as the sole basis for diagnosis, triage, or treatment decisions.